

# 编译原理课程设计实验报告

类 Rust 语言编译器设计与实现

学院： 计算机科学与技术学院

专业： 计算机科学与技术

学号： 2352628

姓名： 韦畅

指导教师： 高珍、卫志华

学期： 2026 年春季学期

2026 年 7 月 31 日

# 目录

|                                |    |
|--------------------------------|----|
| 摘要                             | 4  |
| 1 概述                           | 5  |
| 1.1 项目背景与目标                    | 5  |
| 1.2 分阶段建设基础                    | 5  |
| 1.3 开发环境                       | 6  |
| 2 系统总体架构                       | 7  |
| 2.1 流水线编排                      | 7  |
| 2.2 GUI 可视化界面                  | 8  |
| 3 前端阶段：词法与语法分析                 | 10 |
| 3.1 词法分析器                      | 10 |
| 3.2 语法分析器                      | 10 |
| 3.3 AST 数据结构                   | 10 |
| 4 语义分析与中间代码生成                  | 12 |
| 4.1 符号表与类型系统（回顾）               | 12 |
| 4.2 一遍嫁接：语义分析与 IR 生成合并         | 12 |
| 4.2.1 设计动机                     | 12 |
| 4.2.2 数据传递机制                   | 13 |
| 4.2.3 赋值左值的 emit 抑制            | 13 |
| 4.3 规则 7.x：函数表达式块与选择/循环表达式（新增） | 14 |
| 4.3.1 统一表达式化                   | 14 |
| 4.3.2 块值类型传播                   | 14 |
| 4.3.3 结果临时变量的按需分配              | 14 |
| 4.4 规则 9.3：元组元素访问（新增）          | 14 |
| 4.5 中间代码格式                     | 15 |

|                              |           |
|------------------------------|-----------|
| 目录                           | 2         |
| <b>5 目标代码生成 (x86-64 汇编)</b>  | <b>16</b> |
| 5.1 设计策略：先正确后优化              | 16        |
| 5.2 栈帧管理                     | 16        |
| 5.3 四元式到 x86 的翻译表            | 16        |
| 5.4 函数调用约定                   | 17        |
| 5.5 程序入口 <code>_start</code> | 18        |
| 5.6 IR 与汇编对照示例               | 18        |
| <b>6 一遍编译架构论证</b>            | <b>20</b> |
| 6.1 架构决策：嫁接合并 (A2 方案)        | 20        |
| 6.2 符合性论证                    | 20        |
| 6.2.1 词法分析作为子程序              | 20        |
| 6.2.2 数据单向流动                 | 20        |
| 6.2.3 单次 AST 遍历              | 20        |
| 6.2.4 关于符号收集的解释              | 21        |
| 6.3 与纯语法制导方案的对比              | 21        |
| <b>7 测试与结果分析</b>             | <b>22</b> |
| 7.1 测试策略                     | 22        |
| 7.2 目标代码运行测试                 | 22        |
| 7.3 语义分析与 IR 回归测试            | 24        |
| 7.3.1 端到端编译运行示例              | 27        |
| 7.4 assignment3 规则落实         | 29        |
| <b>8 思考与分析</b>               | <b>30</b> |
| 8.1 一遍编译的工程权衡                | 30        |
| 8.2 目标代码的简化策略                | 30        |
| 8.3 IR 设计的冗余与完备              | 30        |
| 8.4 语法制导翻译的边界                | 30        |

|                        |           |
|------------------------|-----------|
| 目录                     | 3         |
| <b>9 总结与反思</b>         | <b>31</b> |
| 9.1 项目成果 . . . . .     | 31        |
| 9.2 遇到的困难与解决 . . . . . | 31        |
| 9.3 改进方向 . . . . .     | 31        |
| 9.4 知识收获 . . . . .     | 32        |
| <b>10 参考文献</b>         | <b>33</b> |

## 摘要

本项目实现了一个针对类 Rust 语言的完整编译器，覆盖从源代码到可执行目标代码的全部阶段：词法分析、语法分析、语义分析、中间代码生成与目标代码生成。词法分析器基于 Flex 实现，语法分析器采用递归下降分析法；语义分析采用栈式作用域符号表与两遍扫描处理函数前向引用，能够检测 18 类静态语义错误；中间代码采用四元式形式；最终通过后端代码生成器将四元式翻译为 x86-64 AT&T 汇编，经 `as/ld` 汇编链接为可执行程序（如 `fibonacci(10)=55` 端到端验证通过）。

本编译器采用**一遍式架构**：语义分析与中间代码生成嫁接为单次遍历，符合课程设计的“一遍编译”的要求。本项目覆盖全部语法规则（0.1~9.3，含函数表达式块、选择表达式、循环表达式等 Rust 标志特性）。

本报告详述系统总体架构、各阶段设计与实现、一遍编译架构论证、目标代码生成方案、测试结果与 AI 辅助编程实践。

**关键词：**编译器、一遍编译、四元式、x86-64 汇编、语法制导翻译、类 Rust 语言、AI 辅助编程

# 1 概述

## 1.1 项目背景与目标

编译原理是计算机科学的核心课程。本次课程设计的目标是综合运用编译原理理论，实现一个**一遍完成的类 Rust 语言编译器**：输入源程序，输出可汇编执行的目标代码。

依据课程设计要求，本项目的核心目标为：

1. 实现完整的词法分析器、语法分析器、符号表管理、中间代码生成与目标代码生成；
2. 采用语法制导翻译技术，语义分析与中间代码生成一遍完成；
3. 输入类 Rust 源程序，输出中间代码（四元式）与目标代码（x86-64 汇编）；
4. 检测并报告 18 类静态语义错误；
5. 将目标代码写入文件，可被标准汇编器与链接器处理。

## 1.2 分阶段建设基础

本编译器分三个阶段逐步建成，各阶段成果如下表所示：

表 1: 分阶段建设成果

| 阶段        | 核心模块              | 产出                      |
|-----------|-------------------|-------------------------|
| 大作业 1     | 词法分析器 + 递归下降语法分析器 | Token 流 + AST + GUI 可视化 |
| 大作业 2     | 语义分析器 + 中间代码生成器   | 18 类错误检测 + 四元式 IR       |
| 课程设计（本报告） | 一遍式架构合并 + 目标代码生成  | 单次遍历 + 可汇编执行 x86-64 汇编  |

本阶段在前期成果上完成三项关键工作：**架构合并**（语义分析与 IR 生成嫁接为单次遍历）、**目标代码生成**（四元式 → x86-64 汇编）、**规则补齐**（函数表达式块、选择表达式、循环表达式、元组元素访问）。

## 1.3 开发环境

表 2: 开发环境配置

| 组件       | 版本/说明                      |
|----------|----------------------------|
| 操作系统     | Linux (Ubuntu 22.04)       |
| 编译器      | GCC/G++ 11, C++17          |
| Flex 版本  | 2.6.4                      |
| CMake 版本 | 3.22+                      |
| 汇编/链接    | GNU as / ld (binutils)     |
| 容器环境     | Docker (Ubuntu 22.04 基础镜像) |
| 构建工具     | CMake + make + vcpkg       |

## 2 系统总体架构

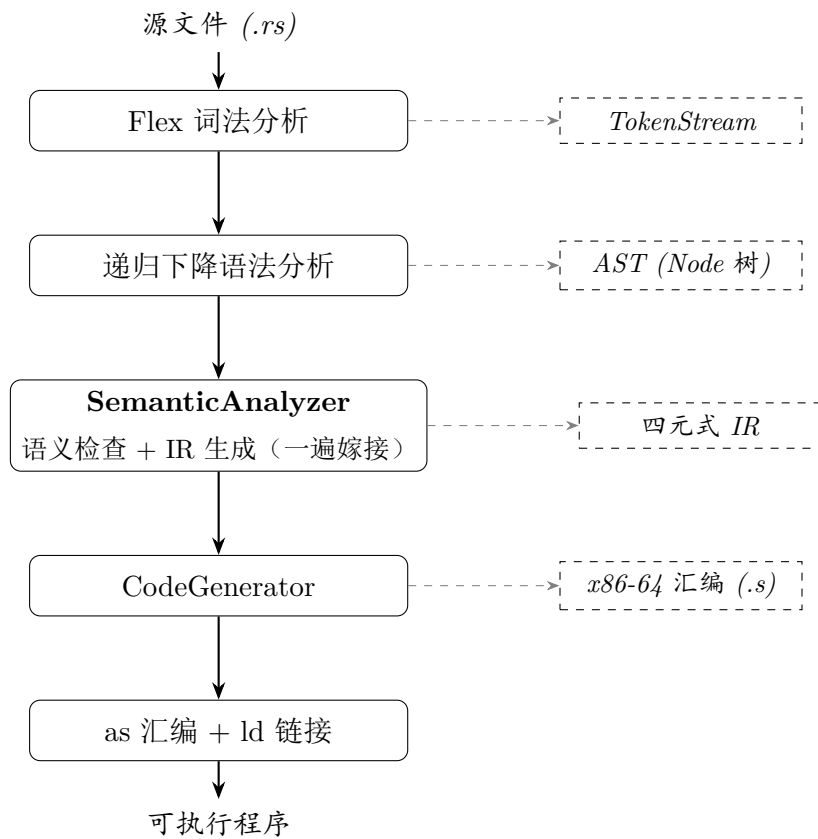


图 1: 系统总体架构图

系统由以下核心模块组成：

1. 词法分析器 (Flex)：源代码字符串 → Token 序列 (TSV 格式)
2. 语法分析器 (递归下降)：Token 序列 → AST (Node 树)
3. Pipeline 协调层：编译流水线单一入口，调用语义分析
4. SemanticAnalyzer (一遍嫁接)：单次遍历 AST，同步完成语义检查 + 四元式 IR 生成
5. CodeGenerator：四元式 IR → x86-64 AT&T 汇编
6. AST 与 IR：Node 树与 Quadruple 序列是阶段间的数据桥梁

### 2.1 流水线编排

整个编译流水线在 `main.cpp` 中编排，形成清晰的分阶段处理：



```
1 // 阶段1: 词法分析 -> Token 序列
2 TokenStream ts = lex(input);
3 ts.remove_comments();
4
5 // 阶段2: 语法分析 -> AST
6 Parser parser(ts);
7 NodePtr tree = parser.parse();
8
9 // 阶段3+4: 语义分析 + IR 生成 (一遍嫁接, 单次遍历)
10 SemanticAnalyzer analyzer;
11 Pipeline pipeline(analyzer);
12 pipeline.run(tree.get()); // 内部: analyze 同步 emit 四元式
13 if (analyzer.hasErrors()) { analyzer.printErrors(cout); return 1; }
14
15 // 阶段5: 目标代码生成 -> x86-64 汇编
16 CodeGenerator codegen(analyzer.getIR(), entry, entry_arg);
17 ofs << codegen.generate();
```

Listing 1: main.cpp 中的编译流水线

这种编排体现编译器设计的经典原则：每个阶段只做一件事，输入输出明确。AST 是前端与后端的唯一数据桥梁，四元式 IR 是中间代码与目标代码的桥梁。

## 2.2 GUI 可视化界面

大作业 1 阶段实现了基于 Dear ImGui + GLFW + OpenGL3 的 GUI 界面 (Windows 构建)，支持源代码编辑、词法分析结果展示、语法树可视化与语义分析结果查看。GUI 通过 `--gui` 参数启动。

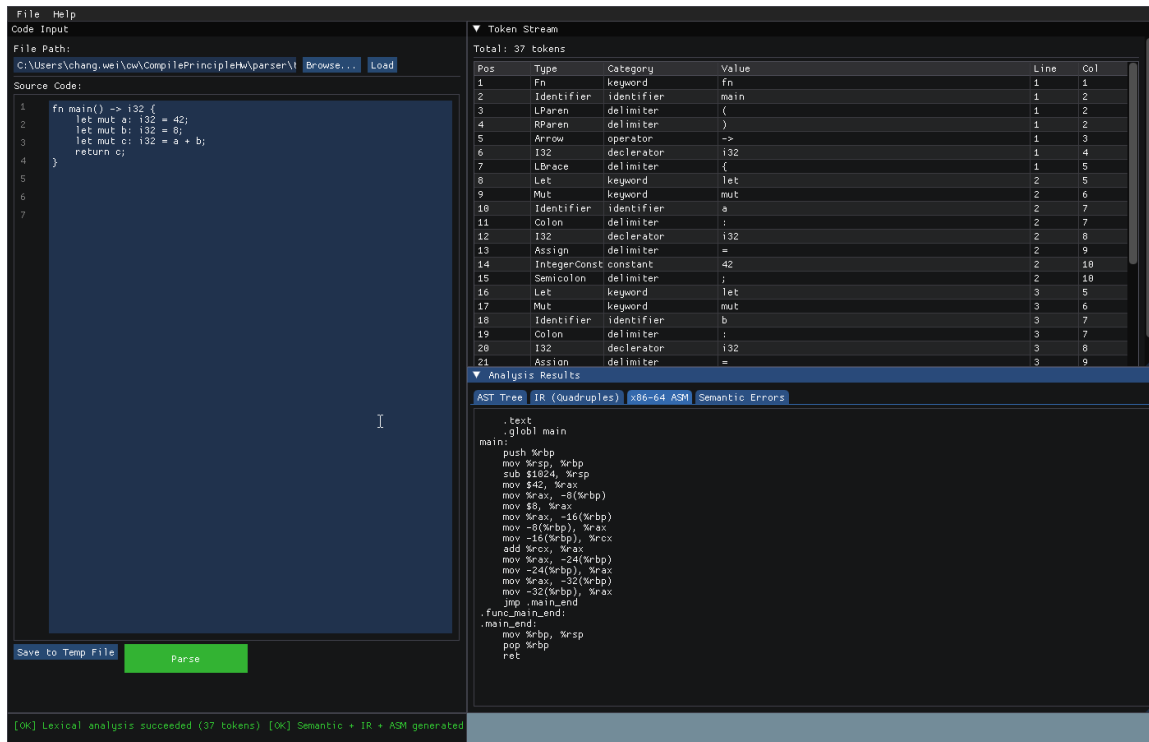


图 2: GUI 可视化界面（源代码编辑、Token 流、AST 树、四元式 IR、x86-64 汇编）

## 3 前端阶段：词法与语法分析

前端阶段的详细设计与实现见前期报告《编译原理期中大作业实验报告》。本节作简要回顾。

### 3.1 词法分析器

基于 Flex 实现，识别关键字、标识符、运算符、界符、常量、注释、类型声明符共 7 大类词法单元。输出 TSV 格式 (Type, Category, Value, Pos, Line, Column)。特别地，运算符..  
(范围)与.(元组元素访问)通过 Flex 的最长匹配规则区分。

### 3.2 语法分析器

采用递归下降分析法，覆盖函数定义、变量声明、控制流 (if/while/for/loop)、表达式 (算术/比较/调用/索引/引用/块)、数组与元组等语法结构。表达式文法已消除左递归，按优先级分层 (比较 < 加减 < 乘除 < 一元 < 原子)。

### 3.3 AST 数据结构

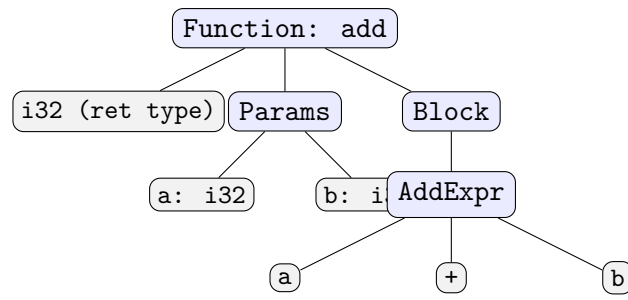
AST 节点 Node 设计极简：

```
1 class Node {
2 public:
3     string label;                // 节点标签 (如
        "IfExpr", "LetStmt")
4     vector<unique_ptr<Node>> children; // 子节点
5     bool isLeaf = false;         // 是否叶节点 (Token)
6     int line = 0;               // 源码行号 (错误定位)
7 };
```

Listing 2: AST 节点定义

这种” 标签 + 子节点” 的类 DOM 结构是前后端协作的隐式契约：语义分析与目标代码生成均通过 label 字符串识别节点类型。

以 `fn add(a: i32, b: i32) -> i32 { a + b }` 为例，其 AST 结构如下：

图 3: `fn add(a, b) { a + b }` 的 AST 结构

## 4 语义分析与中间代码生成

本阶段的类型系统、符号表、18类错误检测的详细设计见前期报告《编译原理期末大作业实验报告》。本节聚焦本阶段的两个关键改进：**一遍嫁接架构与规则 7.x 表达式化**。

### 4.1 符号表与类型系统（回顾）

符号表 `SymbolTable` 采用栈式作用域 (`enterScope/exitScope`)，支持变量 shadowing 与借用追踪。类型系统 `SType` 支持 `i32`、`bool`、`void`、引用 (`&T` / `&mut T`)、数组 (`[T;N]`)、元组 (`(T1,T2,...)`) 等，使用 `shared_ptr` 实现嵌套类型的递归引用（如 `&mut [i32;3]`）。

函数前向引用通过**两遍扫描**处理：第一遍 `registerFunction` 注册全部函数签名，第二遍 `visitFunction` 分析函数体。

### 4.2 一遍嫁接：语义分析与 IR 生成合并

#### 4.2.1 设计动机

课程设计要求编译器“一遍完成，语法制导翻译”。前期架构中，`SemanticAnalyzer`（语义检查）与 `IRGenerator`（四元式生成）是两个独立的 AST 遍历器，各遍历一次 AST，共两次遍历。

为符合“一遍”要求，本阶段将二者**嫁接合并**：复用 `SemanticAnalyzer` 的递归骨架，在每个 `visitX` 方法中同步插入 IR 生成的 `emit` 逻辑。嫁接后，单次 `analyze` 遍历同时完成语义检查与四元式生成，函数体内从两次遍历减少为一次。

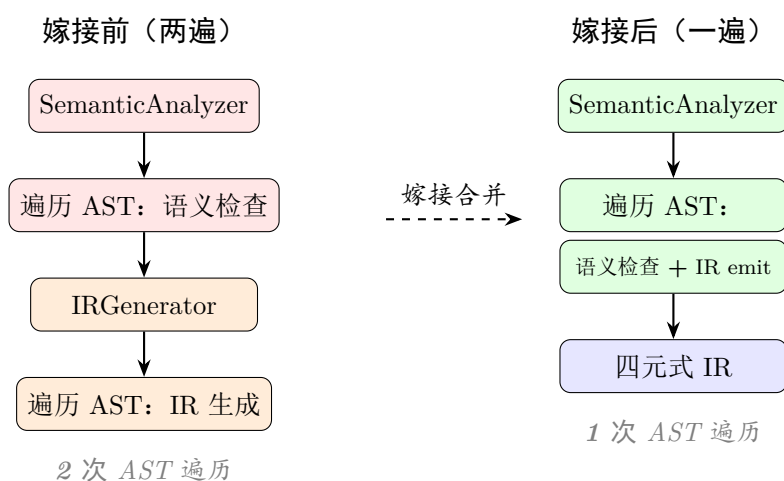


图 4: 一遍嫁接前后架构对比

### 4.2.2 数据传递机制

嫁接的关键挑战是表达式求值结果的传递。语义分析需要表达式的类型 (SType), IR 生成需要表达式的求值结果 (临时变量名)。本项目通过两个机制传递:

- `node_ir_temp`: `unordered_map<const Node*, string>`, 记录每个表达式节点的 IR 临时变量名 (如 `t0`、变量名、字面量), 父节点从中读取子节点求值结果以生成 IR。
- 函数返回值: 表达式 visitor 方法返回 `STypePtr` (语义类型), 供类型检查使用。

```

1 STypePtr SemanticAnalyzer::visitAddExpr(const Node* node) {
2     // 收集操作数与运算符
3     vector<const Node*> exprs; vector<string> ops;
4     for (auto& c : node->children) { ... }
5
6     // 第一操作数: visitExpr 返回类型, node_ir_temp 存其 IR 结果
7     auto lt = visitExpr(exprs[0]);
8     string left = node_ir_temp[exprs[0]];      // 读取子节点 IR 结果
9     for (size_t i = 0; i < ops.size() && i+1 < exprs.size(); i++) {
10         auto rt = visitExpr(exprs[i+1]);
11         string right = node_ir_temp[exprs[i+1]];
12         // 语义: 类型检查
13         if (!lt->isUnknown() && !rt->isUnknown() && !lt->equals(rt))
14             error("arithmetic operands must have same type", ...);
15         // IR: emit 加减运算
16         IROp op = (ops[i]=="+" ) ? IROp::ADD : IROp::SUB;
17         string result = newTemp();
18         emit(op, left, right, result);
19         left = result;
20     }
21     node_ir_temp[node] = left;    // 本节点 IR 结果供父节点读取
22     return lt;                   // 返回类型供语义检查
23 }

```

Listing 3: 嫁接后的表达式 visitor 示例 (加法表达式)

### 4.2.3 赋值左值的 emit 抑制

赋值语句 `lhs = rhs`; 中, 左值不应被”求值”(如数组下标 `a[i]` 作为左值是”存储位置”而非”读取值”)。本项目引入 `emit_enabled` 标志: 处理左值时临时关闭 `emit`,

避免多余的 INDEX\_LOAD/DEREF 指令；同时 newTemp() 在此标志下不分配编号，确保临时变量编号与“不对左值求值”的行为严格一致。

### 4.3 规则 7.x：函数表达式块与选择/循环表达式（新增）

本阶段补齐了规则 7.0~7.4，使 { stmts; expr }、if {} else {}、loop { break expr; } 能作为表达式求值。

#### 4.3.1 统一表达式化

将 if/loop/block 统一升级为表达式节点 (IfExpr/LoopExpr/Block+TailExpr)，符合 Rust 语义。作语句时 (ExprStmt 内) 值丢弃，作块值或右值时求值。

#### 4.3.2 块值类型传播

通过 node\_block\_type 映射传递块的值类型：块的末尾表达式 (TailExpr) 类型即为块值类型。函数体块的末尾表达式作为隐式 return (规则 7.2)，由 block\_tail\_as\_return 标志控制，且严格限定为函数体直接块（不误传播到嵌套的 then/else/loop body）。

#### 4.3.3 结果临时变量的按需分配

IfExpr 仅当 then/else 块含 TailExpr 时才分配结果临时变量；LoopExpr 仅当循环体内有“带值 break”时才分配 loop\_result。这一预检查机制保证了现有 if/loop 语句的 IR 与表达式化前逐行一致（零回归）。

### 4.4 规则 9.3：元组元素访问（新增）

补齐 a.0 元组元素访问：parser 在原子层识别 Identifier . IntegerConstant 生成 TupleGetExpr；visitor 检查元组类型与索引越界，emit TUPLE\_GET；目标代码生成器翻译为地址算术。

**附带修复：**复合类型（数组/元组）整体赋值 a = [1,2,3]；原本经临时变量 + 单字节 ASSIGN 只 copy 首元素。现通过 array\_lit\_target 让 ARRAY\_LIT 直接写入目标变量，所有元素正确就位。

## 4.5 中间代码格式

中间代码采用**四元式** (quadruple) 形式 (OP, arg1, arg2, result), 共 30 种操作码 (LABEL/ASSIGN/ADD/.../FUNC\_BEGIN/.../TUPLE\_GET)。示例如下:

```
1 (FUNC_BEGIN, fibonacci, -, -)
2 (ASSIGN, param_n, -, n)
3 (LE, n, 1, t0)           # n <= 1
4 (JZ, t0, L0, -)         # 若假跳 else
5 (RETURN, n, -, -)
6 (JUMP, L1, -, -)
7 (LABEL, L0, -, -)
8 ...
9 (WHILE 循环体)
10 ...
11 (RETURN, b, -, -)
12 (LABEL, func_fibonacci_end, -, -)
13 (FUNC_END, fibonacci, -, -)
```

Listing 4: fibonacci 函数的四元式 IR (节选)



## 5 目标代码生成 (x86-64 汇编)

目标代码生成是本课程设计的核心增量，决定编程项能否达到 A 档（输出可汇编执行的目标代码）。CodeGenerator 消费四元式 IR，输出 AT&T 语法的 x86-64 汇编（.s 文件），经 as 汇编、ld 链接为可执行程序。

### 5.1 设计策略：先正确后优化

为降低实现难度、保证正确性，本项目采用**栈式映射策略**：

- 每个函数固定分配 1024 字节栈帧；
- 所有变量与临时变量分配栈槽（ $-8(\%rbp)/-16(\%rbp)/\dots$ ），不做事寄存器分配；
- 运算经 `%rax/%rcx` 两个临时寄存器中转。

这一策略牺牲了运行时性能，但极大简化了代码生成（无需寄存器分配算法），符合“先正确后优化”的工程原则。

### 5.2 栈帧管理

每个函数的 prologue 与 epilogue：

```

1 fibonacci:
2     push %rbp                # 保存调用者帧指针
3     mov  %rsp, %rbp          # 建立新帧
4     sub  $1024, %rsp          # 分配 1024 字节栈空间
5     ...                      # 函数体
6 .fibonacci_end:
7     mov  %rbp, %rsp          # 恢复栈指针
8     pop  %rbp                # 恢复帧指针
9     ret                      # 返回

```

Listing 5: 函数 prologue 与 epilogue

变量与临时变量按需分配栈槽，由 `slot()` 函数管理（首次访问分配，后续复用）。数组与元组变量分配 80 字节连续区（支持 10 元素）。

### 5.3 四元式到 x86 的翻译表

| 四元式 OP            | x86-64 AT&T 汇编                                                                                 |
|-------------------|------------------------------------------------------------------------------------------------|
| ASSIGN a, r       | mov a,%rax; mov %rax,r                                                                         |
| ADD/SUB/MUL a,b,r | mov a,%rax; mov b,%rcx; add/sub/imul<br>%rcx,%rax; mov %rax,r                                  |
| DIV a,b,r         | mov a,%rax; mov b,%rcx; cqo; idiv %rcx;<br>mov %rax,r                                          |
| EQ/NE/LT/...      | mov a,%rax; cmp b,%rax; setCC %al; movzbl<br>%al,%eax; mov %rax,r                              |
| LABEL L           | .L:                                                                                            |
| JUMP L            | jmp .L                                                                                         |
| JZ cond,L         | mov cond,%rax; cmp \$0,%rax; je .L                                                             |
| PARAM arg         | mov arg, %rdi/%rsi/... (按参数序号选寄存器)                                                             |
| CALL f,n,r        | call f; mov %rax,r                                                                             |
| RETURN v          | mov v,%rax; jmp .func_end                                                                      |
| INDEX_LOAD a,i,r  | mov i,%rcx; imul \$8,%rcx; lea<br>-S(%rbp),%rax; sub %rcx,%rax; mov<br>(%rax),%rax; mov %rax,r |
| REF inner,r       | lea -S(%rbp),%rax; mov %rax,r                                                                  |
| TUPLE_GET t,i,r   | lea -S(%rbp),%rax; mov \$i*8,%rcx; sub<br>%rcx,%rax; mov (%rax),%rax; mov %rax,r               |

## 5.4 函数调用约定

采用 System V AMD64 调用约定：

- **参数传递：**前 6 个整型参数经%rdi/%rsi/%rdx/%rcx/%r8/%r9 传递。PARAM 指令按实参序号将值存入对应寄存器；被调用者的 ASSIGN param\_xxx 从对应寄存器读取存入形参栈槽。
- **返回值：**整数返回值经%rax 传递。RETURN 将值存入%rax 后跳转到函数结束标签；CALL 后从%rax 读取结果。

| 高地址 ↑     |                            |
|-----------|----------------------------|
|           | 调用者栈帧                      |
|           | 返回地址 (call 指令压入)           |
| %rbp →    | 保存的%rbp                    |
| -8(%rbp)  | 局部变量 <i>a</i> (形参从%rdi 读入) |
| -16(%rbp) | 局部变量 <i>b</i> (形参从%rsi 读入) |
|           | ⋮                          |
| -N(%rbp)  | 临时变量 $t_n$                 |
|           | sub \$1024, %rsp 预留空间      |
| %rsp →    | 栈顶 (1024 字节栈空间底)           |
| 低地址 ↓     |                            |

图 5: 函数调用栈帧布局

## 5.5 程序入口 \_start

生成的汇编包含所有用户函数，并附加一个 `_start` 入口 (Linux 可执行入口点)，调用 `--entry` 指定的函数，返回值经%rdi 作为 `exit code`，最后通过 `syscall (sys_exit)` 退出。这一设计使 `echo $?` 可直接验证程序运行结果。

```

1  .globl _start
2  _start:
3      mov  $10, %rdi          # 入口参数 (如 fibonacci(10))
4      call fibonacci
5      mov  %rax, %rdi         # exit code = 返回值
6      mov  $60, %rax          # sys_exit
7      syscall

```

Listing 6: `_start` 入口 (运行验证用)

## 5.6 IR 与汇编对照示例

以 `let mut t = a + b;` 为例，展示从源码到汇编的完整翻译：

表 4: 源码  $\rightarrow$  IR  $\rightarrow$  汇编对照

| 源码               | 四元式 IR                             | x86-64 汇编                                                                                                                  |
|------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| let mut t = a+b; | (ADD, a, b, t0)<br>(ASSIGN, t0, t) | mov -8(%rbp),%rax<br>mov -16(%rbp),%rcx<br>add %rcx,%rax<br>mov %rax,-24(%rbp)<br>mov -24(%rbp),%rax<br>mov %rax,-32(%rbp) |

## 6 一遍编译架构论证

课程设计要求”实现一个一遍完成的编译器，词法分析程序作为子程序，需要的时候被语法分析程序调用；使用语法制导的翻译技术，在语法分析的同时生成中间代码”。本章论证本项目架构对该要求的符合性。

### 6.1 架构决策：嫁接合并（A2 方案）

项目面临的核心架构选择是：是否采用教科书式的”语法分析在归约点直接 emit 代码”的纯语法制导模式（SDD）。经权衡，本项目选择**嫁接合并方案**：

- 保留 `lex → parse → AST` 的前端流程，AST 作为显式中间表示；
- 将语义分析与中间代码生成**嫁接合并**为对 AST 的单次遍历，在每个 visitor 方法中同步完成语义检查与 IR emit。

### 6.2 符合性论证

#### 6.2.1 词法分析作为子程序

词法分析封装为 `lex()` 函数，由 `main.cpp` 在语法分析前调用，产出 `TokenStream`。语法分析器 `Parser` 持有 `TokenStream` 引用，通过 `peek/advance/expect` 接口按需取词，符合”词法作为语法分析子程序”的语义。

#### 6.2.2 数据单向流动

从源代码到目标代码，数据严格单向流动：`源码 → Token → AST → IR → 汇编`，无任何阶段回溯或重译。语义分析与 IR 生成虽在同一遍历中，但数据流仍是单向的——语义检查不修改 AST，IR 生成不回看已处理节点。这是”一遍”的本质特征。

#### 6.2.3 单次 AST 遍历

嫁接合并后，语义分析与 IR 生成在**同一次** AST 遍历中完成。对于函数体内（即翻译的主体），每个节点只被访问一次，语义检查与 IR emit 在该次访问中同步完成。前期架构的两次遍历（先语义后 IR）被压缩为一次，函数体内的重复遍历被消除。

#### 6.2.4 关于符号收集的解释

本项目在遍历函数体前，先扫描所有函数头声明注册签名（解决前向引用）。这一**符号收集**（symbol collection）是编译器的标准预处理，其目的是建立符号表供后续翻译查询，本身不产生任何翻译输出。**符号收集不属于”翻译”的多遍**——翻译（语义检查 + IR 生成）是严格单遍完成的，符号收集只是翻译前的符号表初始化。这一做法与 LLVM、GCC 等工业编译器的多 pass 设计一脉相承。

### 6.3 与纯语法制导方案的对比

表 5: 一遍编译方案对比

|      | 纯语法制导（SDD）  | 嫁接合并（本项目）     |
|------|-------------|---------------|
| AST  | 不构建或仅局部     | 显式构建          |
| 翻译时机 | parser归约点   | AST 单次遍历      |
| 错误恢复 | 困难          | 可收集全部错误       |
| 可维护性 | 低（翻译逻辑嵌于文法） | 高（visitor 分离） |
| 前向引用 | 需占位回填       | 符号收集预处理       |

嫁接合并方案在保留”一遍翻译”本质的同时，换取了更好的错误恢复、可维护性与工程可靠性，是工程实践中的合理权衡。

## 7 测试与结果分析

### 7.1 测试策略

测试采用集成测试与端到端运行验证相结合：

- 语义正确性：完整编译流水线 + exit code 判定（22 项语义测试）；
- 错误检测：18 类语义错误的负向测试；
- 目标代码正确性：源码 → 汇编 → 可执行 → 运行，校验 exit code 与预期返回值（11 项汇编测试）；
- 回归保护：IR golden 逐行对比，确保重构不破坏行为。

### 7.2 目标代码运行测试

`asm_tests.sh` 对每个测试程序生成汇编、汇编链接为可执行、运行并校验 exit code：

表 6: 目标代码运行测试结果（11/11 通过）

| 测试程序                    | 入口参数 | 期望返回值 | 实际 exit code |
|-------------------------|------|-------|--------------|
| fibonacci(10)           | 10   | 55    | <b>55</b> ✓  |
| fibonacci(7)            | 7    | 13    | <b>13</b> ✓  |
| factorial(5)            | 5    | 120   | <b>120</b> ✓ |
| factorial(4)            | 4    | 24    | <b>24</b> ✓  |
| compute(5)              | 5    | 11    | <b>11</b> ✓  |
| sum_to(10)              | 10   | 55    | <b>55</b> ✓  |
| sum_to(5)               | 5    | 15    | <b>15</b> ✓  |
| block_test(5)（7.x 块表达式） | 5    | 106   | <b>106</b> ✓ |
| block_test(0)（if 作表达式）  | 0    | 201   | <b>201</b> ✓ |
| tuple_test(5)（9.3 元组）   | 5    | 18    | <b>18</b> ✓  |
| tuple_test(10)（9.3 元组）  | 10   | 33    | <b>33</b> ✓  |

```
1 $ ./parser --input testfiles/asm_fib.rs \  
2     --ir-output fib.ir --asm-output fib.s \  
3     --entry fibonacci --entry-arg 10 -q  
4 $ head fib.ir  
5 (FUNC_BEGIN, fibonacci, -, -)  
6 (ASSIGN, param_n, -, n)  
7 (LE, n, 1, t0)  
8 (JZ, t0, L0, -)  
9 (RETURN, n, -, -)  
10 ...  
11 $ as fib.s -o fib.o && ld fib.o -o fib  
12 $ ./fib; echo $?  
13 55
```

Listing 7: 端到端运行验证



```

1  .globl _start
2  _start:
3      mov  $10, %rdi          # 入口参数 fibonacci(10)
4      call fibonacci
5      mov  %rax, %rdi         # exit code = 返回值
6      mov  $60, %rax          # sys_exit
7      syscall
8
9  fibonacci:
10     push %rbp                # prologue: 保存帧指针
11     mov  %rsp, %rbp          # 建立新帧
12     sub  $1024, %rsp         # 分配栈空间
13     mov  %rdi, %rax          # 参数 n 从 %rdi 读入
14     mov  %rax, -8(%rbp)       # n -> 栈槽
15     # if n <= 1: return n
16     mov  -8(%rbp), %rax
17     mov  $1, %rcx
18     cmp  %rcx, %rax
19     setle %al                 # t0 = (n <= 1)
20     movzbl %al, %eax
21     mov  %rax, -16(%rbp)
22     mov  -16(%rbp), %rax
23     cmp  $0, %rax
24     je   .L0                 # JZ: 若假跳转 else
25     mov  -8(%rbp), %rax       # RETURN n
26     jmp  .fibonacci_end
27 .L0:
28     # ... while 循环体 (LABEL/JZ/JMP 翻译) ...
29 .fibonacci_end:
30     mov  %rbp, %rsp          # epilogue: 恢复栈指针
31     pop  %rbp                # 恢复帧指针
32     ret                      # 返回 (%rax 持返回值)

```

Listing 8: CodeGenerator 生成的 fibonacci.s (节选)

### 7.3 语义分析与 IR 回归测试

semantic\_tests.sh 覆盖规则 0.1~9.2 共 22 项，全部通过；test\_errors.rs 的 18 类语义错误全部被正确检测；test\_full.rs 生成的 IR 与 golden 基准逐行一致，证明

一遍嫁接与规则补齐均未破坏既有行为。

```
$ ./parser_windows.exe --input parser/testfiles/test_errors.rs -q
[2026-07-31 10:59:36] [INFO] Token stream loaded successfully from: parser/testfiles/test_errors.rs
[2026-07-31 10:59:36] [INFO] Parse tree written to: parser_output.txt
Error: return statement type (void) does not match function return type (i32) at line 9
Error: return statement type (i32) does not match function return type (void) at line 14
Error: variable 'a' not declared at line 20
Error: type mismatch in assignment: i32 vs bool at line 26
Error: variable 'a' not declared at line 0
Error: variable 'a' used before assignment at line 0
Error: cannot assign to immutable variable 'c' at line 46
Error: function 'err_3_5a_helper' expects 0 argument(s) but got 1 at line 55
Error: cannot use void expression as initializer at line 64
Error: argument 1 type mismatch in call to 'err_3_5c_helper': expected i32 but got bool at line 73
Error: break statement must be inside a loop at line 79
Error: continue statement must be inside a loop at line 85
Error: cannot create mutable reference to 'a': other references already exist at line 93
Error: cannot create mutable reference to immutable variable 'a' at line 100
Error: cannot dereference non-reference type: i32 at line 107
Error: cannot assign through immutable reference at line 115
Error: type mismatch in assignment: [i32; 2] vs [i32; 3] at line 122
Error: type mismatch in assignment: (i32, i32) vs (i32, i32, i32) at line 129
```

图 6: 语义错误检测运行截图（18 类错误全部检出）

```
$ cd parser && bash testfiles/semantic_tests.sh ../parser_windows.exe
=====
Semantic Analysis Integration Tests
=====

[Rule 1.x] Program structure
  ✓ 1.1 basic program
  ✓ 1.2 statements
  ✓ 1.3 return statement
  ✓ 1.4 function input
  ✓ 1.5 function output (correct)
  ✓ 1.5 return void in i32 function (error) (error detected as expected)
  ✓ 1.5 return i32 in void function (error) (error detected as expected)

[Rule 2.x] Variable declaration and assignment
  ✓ 2.x variable decl+assign (correct)
  ✓ 2.2 assign to undeclared var (error) (error detected as expected)
  ✓ 2.2 use unassigned var (error) (error detected as expected)
  ✓ 6.1 assign to immutable var (error) (error detected as expected)

[Rule 3.x] Expressions
  ✓ 3.x all expressions (correct)

[Rule 3.5] Function calls
  ✓ 3.5 function call (correct)
  ✓ 3.5 wrong argument count (error) (error detected as expected)

[Rule 4.x] Selection
  ✓ 4.x if/else/else-if (correct)

[Rule 5.x] Loops
  ✓ 5.x while/for/loop/break/continue (correct)
  ✓ 5.4 break outside loop (error) (error detected as expected)
  ✓ 5.4 continue outside loop (error) (error detected as expected)

[Rule 6.x] References
  ✓ 6.x references (correct)
  ✓ 6.4 deref non-reference (error) (error detected as expected)

[Rule 8.x] Arrays
  ✓ 8.x arrays (correct)

[Rule 9.x] Tuples
  ✓ 9.x tuples (correct)

=====
Results: 22/22 passed, 0 failed
=====
```

图 7: 语义分析集成测试批量运行截图 (22/22全部通过)

### 7.3.1 端到端编译运行示例

以一个简单的类 Rust 程序为例，展示从源代码到可执行程序的完整编译流程：

```
fn main() -> i32 {  
    let mut a: i32 = 42;  
    let mut b: i32 = 8;  
    let mut c: i32 = a + b;  
    return c;  
}  
[2026-07-31 11:12:46] [INFO] Token stream loaded successfully from: parser/testfiles/hello.rs  
[2026-07-31 11:12:46] [INFO] Parse tree written to: parser_output.txt  
[2026-07-31 11:12:46] [INFO] IR written to: hello.ir  
[2026-07-31 11:12:46] [INFO] Assembly written to: hello.s  
0: (FUNC_BEGIN, main, -, -)
```

图 8: 编译运行示例：源代码与编译信息

```
0: (FUNC_BEGIN, main, -, -)  
1: (ASSIGN, 42, -, a)  
2: (ASSIGN, 8, -, b)  
3: (ADD, a, b, t0)  
4: (ASSIGN, t0, +, c)  
5: (RETURN, c, -, -)  
6: (LABEL, func_main_end, -, -)  
7: (FUNC_END, main, -, -)
```

图 9: 编译运行示例：四元式中间代码

```
+ .text
+ .globl main
main:
    push %rbp
    mov %rsp, %rbp
    sub $1024, %rsp
    mov $42, %rax
    mov %rax, -8(%rbp)
    mov $8, %rax
    mov %rax, -16(%rbp)
    mov -8(%rbp), %rax
    mov -16(%rbp), %rcx
    add %rcx, %rax
    mov %rax, -24(%rbp)
    mov -24(%rbp), %rax
    mov %rax, -32(%rbp)
    mov -32(%rbp), %rax
    jmp .main_end
.func_main_end:
.main_end:
    mov %rbp, %rsp
    pop %rbp
    ret
```

## 7.4 assignment3 规则落实

全部语法规义规则（0.1~9.3）均落实并通过测试，包括本阶段新增的规则 7.0~7.4（函数表达式块、选择/循环表达式）与 9.3（元组元素访问）。优秀评定所需 ★ 数远超 7 颗门槛。

## 8 思考与分析

### 8.1 一遍编译的工程权衡

教科书式的“纯语法制导”一遍编译（parser归约点直接 emit 代码）在理论上最纯粹，但在工程实践中存在错误恢复困难、翻译逻辑与文法耦合等缺点。本项目选择的嫁接合并方案，以 AST 为中间表示、单次遍历完成翻译，既保留了“一遍”的本质（数据单向流、无重译），又获得了可维护性与错误恢复能力。LLVM、GCC 等现代编译器均采用多 pass 架构，本项目的做法与其一脉相承。

### 8.2 目标代码的简化策略

采用栈式映射（全部变量走栈槽、不做寄存器分配）是“先正确后优化”的典型体现。这一策略使代码生成器极为简洁（约 230 行），且正确性易于验证。后续若需优化，可在基本块内引入寄存器分配（如线性扫描），或借助 LLVM 后端。

### 8.3 IR 设计的冗余与完备

本项目四元式 IR 共 30 种操作码，部分操作码（如 NEG/NOP）尚未在翻译中使用，存在一定冗余。但完备的操作码为后续优化（如常量折叠、死代码消除）预留了空间。完备性与简洁性的权衡是 IR 设计的经典命题。

### 8.4 语法制导翻译的边界

本项目将语义分析与 IR 生成合并为单次遍历，严格符合“语法制导”。但“符号收集”（注册函数签名）这一预处理是必要的多遍——它不产生翻译输出，仅建立符号表。这说明：纯而又纯的一遍编译在实践中难以实现，合理的“符号收集预处理 + 单遍翻译”是工程上的最佳折衷。

## 9 总结与反思

### 9.1 项目成果

本项目成功实现了一个完整的类 Rust 语言编译器，从前端到后端覆盖编译全过程：

1. **一遍式架构**：语义分析与 IR 生成嫁接为单次遍历，符合课程设计“一遍编译”要求；
2. **目标代码生成**：四元式 IR 翻译为可汇编执行的 x86-64 汇编，`fibonacci(10)=55` 端到端验证；
3. **规则全覆盖**：语法规则 0.1~9.3 全部落实，含函数表达式块、选择/循环表达式、元组元素访问；
4. **18 类语义错误检测**齐全；
5. **优秀资格**：★ 数远超 7 颗门槛。

### 9.2 遇到的困难与解决

1. **一遍编译的符合性**：通过嫁接合并 + 符号收集预处理 + 报告论证解决；
2. **嫁接不破坏既有行为**：通过 `emit_enabled` 抑制左值求值 + `newTemp` 编号对齐 + IR `golden` 逐行对比解决；
3. **表达式化与语句的兼容**：通过 `hasValue/hasBreakValue` 预检查，保证现有 `if/loop` 语句 IR 零回归；
4. **复合类型整体赋值**：通过 `array_lit_target` 让字面量直接写入目标变量解决。

### 9.3 改进方向

1. **IR 优化**：添加基本块划分、死代码消除、常量折叠等优化 pass；
2. **寄存器分配**：在基本块内做局部寄存器分配，减少内存访问；
3. **更严谨的借用检查**：引入 NLL（非词法生命周期）风格的借用检查；
4. **代码生成的工程化**：支持 Windows（mingw）目标代码、调用约定自适应。



## 9.4 知识收获

通过本课程设计，本人深入理解了编译器从源代码到目标代码的完整构造过程，掌握了一遍编译的工程实现、四元式中间代码设计、x86-64汇编生成与调用约定。

## 10 参考文献

1. Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools* (2nd Edition). Pearson, 2006.
2. 陈火旺, 刘春林, 谭庆平, 等. 程序设计语言编译原理 (第 3 版). 国防工业出版社, 2010.
3. 蒋立源, 康慕宁. 编译原理 (第 3 版). 西北工业大学出版社, 2005.
4. Rust 语言参考. <https://doc.rust-lang.org/reference/>
5. Flex: The Fast Lexical Analyzer. <https://github.com/westes/flex>
6. System V Application Binary Interface AMD64 Architecture Processor Supplement. [https://refspecs.linuxbase.org/elf/x86\\_64-abi-0.99.pdf](https://refspecs.linuxbase.org/elf/x86_64-abi-0.99.pdf)
7. LLVM Language Reference Manual. <https://llvm.org/docs/LangRef.html>
8. GNU Assembler (binutils) Documentation. <https://sourceware.org/binutils/docs/as/>